5.2.1. The standard atomic types

The standard *atomic types* can be found in the `<atomic>` header. All operations on such types are atomic, and only operations on these types are atomic in the sense of the language definition, although you can use mutexes to make other operations *appear* atomic. In actual fact, the standard atomic types themselves might use such emulation: they (almost) all have an `is_lock_free()` member function, which allows the user to determine whether operations on a given type are done directly with atomic instructions (`x.is_lock_free()` returns `true`) or done by using a lock internal to the compiler and library (`x.is_lock_free()` returns `false`).

The only type that doesn't provide an `is_lock_free()` member function is `std::atomic_flag`. This type is a really simple Boolean flag, and operations on this type are *required* to be lock-free; once you have a simple lock-free Boolean flag, you can use that to implement a simple lock and thus implement all the other atomic types using that as a basis. When I said *really simple*, I meant it: objects of type `std::atomic_flag` are initialized to clear, and they can then either be queried and set (with the `test_and_set()` member function) or cleared (with the `clear()` member function). That's it: no assignment, no copy construction, no test and clear, no other operations at all.

The remaining atomic types are all accessed through specializations of the `std::atomic<>` class template and are a bit more full-featured but may not be lock-free (as explained previously). On most popular platforms it's expected that the atomic variants of all the built-in types (such as `std::atomic<int>` and `std::atomic<void*>`) are indeed lock-free, but it isn't required. As you'll see shortly, the interface of each specialization reflects the properties of the type; bitwise operations such as `&=` aren't defined for plain pointers, so they aren't defined for atomic pointers either, for example.

In addition to using the `std::atomic<>` class template directly, you can use the set of names shown in table 5.1 to refer to the implementation-supplied atomic types. Because of the history of how atomic types were added to the C++ Standard, these alternative type names may refer either to the corresponding `std::atomic<>` specialization or to a base class of that specialization. Mixing these alternative names with direct naming of `std::atomic<>` specializations in the same program can therefore lead to nonportable code.

Table 5.1. The alternative names for the standard atomic types and their corresponding `std::atomic<>` specializations

Atomic type	Corresponding specialization
atomic_bool	std::atomic<bool>
atomic_char	std::atomic<char>
atomic_schar	std::atomic<signed char>
atomic_uchar	std::atomic<unsigned char>
atomic_int	std::atomic<int>
atomic_uint	std::atomic<unsigned>
atomic_short	std::atomic<short>
atomic_ushort	std::atomic<unsigned short>
atomic_long	std::atomic<long>